Step-by-Step Plan to Build the MVP for Qatar Regulations Chatbot

This plan is designed to deliver a functional MVP within 3 days, leveraging your in-depth knowledge of Agentic AI systems. It focuses on creating a chat-based AI that provides precise, context-specific answers grounded in Qatar government laws and regulations, using Retrieval-Augmented Generation (RAG) to minimize hallucinations and ensure reliability. The plan addresses your implicit concerns: tech stack selection (recommending n8n for speed in a low-code setup, with Python as a fallback for more customization), sourcing official documents, implementing RAG, evaluation for accuracy, and optimizing the stack for best practices.

The plan assumes a total of ~24 working hours across 3 days (e.g., 8 hours/day), broken into phases for efficiency. Prioritize quick iterations—start with a minimal set of documents (e.g., 10-20 key laws) and expand. Use open-source or free-tier tools where possible to keep costs low. If issues arise (e.g., data access), fallback to web scraping or APIs.

Day 1: Research, Data Sourcing, and Setup (Focus: q2, q5 – Sourcing docs and optimal stack)

1. **Research and Identify Official Qatar Legal Sources (1-2 hours)**
    - Search for and compile a list of authoritative sources. Based on web research, the primary source is Al Meezan, Qatar's official legal portal, which hosts all legislations since 1961 in English/Arabic, searchable by year, type (e.g., laws, decrees), and topic. It's comprehensive, timely updated, and CC-BY licensed for use.
    - Secondary sources: Ministry of Commerce and Industry for business regs; Ministry of Justice for general laws; Qatar Open Data Portal for datasets/PDFs; Library of Congress Guide for curated links.
    - Action: Browse Al Meezan and download/export initial documents (e.g., PDFs or HTML of top laws like Data Protection Law No. 13/2016, Commercial Laws). Target 10-20 docs covering common queries (e.g., business setup, labor, privacy). Use tools like Python's BeautifulSoup (if scraping) or manual downloads. If geo-restricted in Qatar, use VPN if needed. Store in a folder for ingestion.
2. **Select and Justify Tech Stack (30 minutes)**
    - Recommendation: Use n8n (low-code workflow automation) as the core for the MVP—it's faster for a 3-day build than full Python coding, supports RAG out-of-the-box (via AI nodes for embedding, vector stores, and agents), and aligns with Agentic workflows (e.g., chaining tools for retrieval + generation). It's open-source, scalable, and has proven templates for RAG chatbots (e.g., from n8n docs and community). Avoids boilerplate code while allowing Python nodes for custom logic.
    - Why not pure Python? While flexible (e.g., with LangChain for RAG and uv for fast virtual env setup), it requires more coding time for plumbing (e.g., integrations). Use Python as a supplement inside n8n if needed.
    - Full stack:
        - Workflow: n8n (for RAG pipeline and Agentic logic).
        - LLM: Grok API (if available via xAI) or OpenAI GPT-4o-mini (cost-effective, accurate for legal queries; free tier for testing).

        - Vector DB: Supabase (free tier, integrates seamlessly with n8n; alternative: Pinecone for scalability).
        - Embeddings: OpenAI embeddings or Sentence Transformers (open-source).
        - UI: Streamlit (Python-based, quick chat interface) or n8n's built-in HTTP triggers for a simple web endpoint.
        - Environment: Use uv (fast Python installer) for any Python dependencies; Docker for n8n if local setup.
        - Why optimal? Based on web benchmarks (e.g., n8n RAG tutorials, Langflow/Flowise comparisons), this stack enables quick prototyping, low hallucinations via grounding, and Agentic features (e.g., multi-step reasoning). It's AI-ready, with built-in error handling.

3. **Set Up Environment and Tools (2-3 hours)**
    - Install n8n: Run locally via npm (npm install -g n8n) or Docker (docker run -it --rm --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n n8nio/n8n).
    - Set up vector DB: Sign up for Supabase (free), create a project, enable vector extension.
    - Install uv: curl -Ls https://github.com/astral-sh/uv/install.sh | sh, then create a virtual env (uv venv).
    - Test integrations: Add n8n nodes for HTTP (data fetch), AI Agent (for RAG), and Supabase. Import a sample RAG template from n8n community (e.g., from blog.n8n.io/rag-chatbot).
    - Gather dependencies: In uv env, install langchain (if needed for custom nodes), requests, beautifulsoup4 for scraping.
4. **Initial Data Ingestion Test (1-2 hours)**
    - In n8n: Create a workflow to fetch docs from Al Meezan (use HTTP node to scrape/search). Chunk text (e.g., 500-1000 chars per chunk via Split node). Generate embeddings (AI Embeddings node). Store in Supabase (Vector Store node).
    - Test with 1-2 docs: Ensure chunks capture context (e.g., article numbers, dates). This addresses q3 (RAG implementation) early.

Day 1 Milestone: Data sources identified, environment ready, sample docs ingested. Demo a basic retrieval.

Day 2: Implement Core RAG and Agentic Logic (Focus: q1, q3 – n8n/Python RAG setup)

5. **Build the RAG Pipeline in n8n (3-4 hours)**
    - Workflow structure:
        - Trigger: HTTP Request (for chat input, e.g., user query like "What are Qatar's data protection rules?").

        - Retrieve: Use Vector Store Query node to fetch top-k (e.g., 5) relevant chunks from Supabase based on query embedding.
        - Augment: Combine retrieved context with prompt (e.g., "Answer precisely using only Qatar laws: [context]. Query: [user_input]. Cite sources.").
        - Generate: AI Agent node with LLM (e.g., OpenAI) to produce response. Add Agentic elements: If query needs multi-step (e.g., compare laws), chain agents (one for retrieval, one for reasoning).
    - Handle precision: Instruct LLM to refuse generalized answers (e.g., "If no Qatar-specific context, say 'No relevant Qatar law found'"). Use reranking if time (e.g., via Cohere node).
    - Integrate Python if needed: Add Code node in n8n for custom scraping or cleaning (e.g., remove boilerplate from Al Meezan HTML).

6. **Expand Data Ingestion (2 hours)**
    - Ingest more docs: Automate fetching from Al Meezan (e.g., query by year/topic). Use n8n's Loop node for batch processing. Aim for coverage of key areas (e.g., business, labor, privacy) based on client needs.
    - Clean data: Remove noise (e.g., headers) to improve retrieval accuracy.
7. **Add Chat Interface (1-2 hours)**
    - Use Streamlit for a simple web app: In Python (via uv env), create a chat UI that calls the n8n workflow via webhook. Script: Import streamlit, requests; display chat history; send query to n8n endpoint; show response with citations.
    - Alternative: n8n's built-in chat trigger if no UI needed (demo via curl/postman).

Day 2 Milestone: End-to-end RAG chat working locally. Test 5-10 sample queries (e.g., "Qatar visa rules") for precision.

Day 3: Evaluate, Refine, and Prepare Demo (Focus: q4 – Reliability and accuracy)

8. **Evaluate Reliability and Accuracy (2-3 hours)**
    - Manual testing: Create 20-30 test queries (e.g., factual, edge cases like "Outdated laws?"). Check for: Accuracy (matches source), Comprehensiveness (covers details), No hallucinations (grounded in retrieved context), Precision (Qatar-specific only).
    - Automated eval: Use RAGAS framework (Python lib, install via uv: uv pip install ragas). Metrics: Faithfulness (response aligns with context), Answer relevance, Context recall. Run on test set—aim for >85% scores. If hallucinations occur, tune prompt (e.g., add "Stick strictly to provided docs").
    - Agentic checks: Test multi-turn chats (e.g., follow-ups) for consistency. Log errors in n8n for debugging.
    - Bias/harm: Manually review for any misinterpretation of laws.
9. **Refine and Optimize (1-2 hours)**

- Improve retrieval: Adjust chunk size, embedding model, or add metadata (e.g., law year) for filtering.
- Scale check: Test with more docs; ensure response time <5s.
- Security: Since Qatar-based, note data privacy (use local/offline if sensitive).

10. **Deploy and Prepare Client Demo (1-2 hours)**
    - Host: Deploy n8n on Vercel/Heroku (free) or local for demo. Share Streamlit app via ngrok for remote access.
    - Demo prep: Record a 5-min video/script showing chat in action, highlighting precision (e.g., citations to Al Meezan). Explain scalability (e.g., add more docs/agents).
    - Backup: If n8n hits limits, pivot to Python-only (LangChain RAG tutorial from medium.com).

Day 3 Milestone: Polished MVP ready. Total: Functional, reliable chatbot demonstrating capability.

This plan is executable, optimal for time constraints, and scalable. If data sourcing blocks (e.g., no API), fallback to web-scraped PDFs. Track progress hourly to adjust.